

Assignment 4: Generative Models with GANs on Adult Tabular Data

Yehonatan Segal, 209359801 | Omer Onn, 318910759

1. Introduction

In this assignment, we implemented and evaluated generative models for tabular data using the Adult dataset. The main goal was to train a standard GAN and a conditional GAN (cGAN), generate synthetic tabular samples, and evaluate both their similarity to the real data and their usefulness for downstream prediction. Since the Adult dataset contains both continuous and categorical features, special attention was given to preprocessing decisions and to the representation of discrete features during generation.

We evaluated the models using two complementary metrics. Detection measures whether a Random Forest classifier can distinguish real samples from synthetic samples; therefore, lower AUC values are better, with 0.5 being ideal. Efficacy measures whether synthetic data can replace real data for predicting income; it is computed as the AUC of a classifier trained on synthetic data divided by the AUC of a classifier trained on real data, both evaluated on the real test set.

The report also includes additional experiments on differentiable categorical generation, mode collapse, and an open-ended modification using feature matching.

The following sections describe the dataset, preprocessing pipeline, model architectures, evaluation metrics, and additional experiments.

2. Dataset and Preprocessing

2.1 Dataset

The Adult dataset was provided in ARFF format, which includes both the data and a header specifying the possible values of discrete features. The dataset contains 32,561 samples and 15 columns. The target variable is income, with two classes: $\leq 50K$ and $> 50K$. The original label distribution is imbalanced: 75.919% of the samples belong to $\leq 50K$ and 24.081% belong to $> 50K$.

The dataset includes 6 numerical features: age, fnlwgt, education-num, capital-gain, capital-loss, and hours-per-week. It also includes 8 categorical features: workclass, education, marital-status, occupation, relationship, race, sex, and native-country.

Missing values were found only in categorical columns: workclass, occupation, and native-country. Since these missing values represent unknown categorical information, we treated them as a dedicated Missing category rather than removing rows.

The dataset summary, missing values, and feature types are shown in Appendix Table A1.

2.2 Train-test split

For evaluation, we created three independent 80%/20% train-test splits using random seeds 42, 123, and 2026. The split was stratified by the income target in order to preserve the original label ratio in both train and test sets. Each training set contains 26,048 samples and each test set contains 6,513 samples. In all three splits, the training set contains 19,775 samples of $\leq 50K$ and 6,273 samples of $> 50K$, preserving the original class distribution.

The exact split statistics for all seeds are reported in Appendix Table A2.

2.3 Preprocessing decisions

All preprocessing transformations were fitted on the training set only and then applied to the corresponding test set. Numerical features were median-imputed and scaled to the range $[-1, 1]$ using MinMax scaling. This range was chosen because the generator uses tanh activation for numerical outputs. Categorical features were imputed using a dedicated Missing category and encoded using one-hot encoding. The target income was also represented as a one-hot vector.

To ensure that all splits have the same representation, the one-hot encoders were initialized using the categories declared in the ARFF header rather than only the categories observed in a specific training split. The final feature vector contains 113 feature dimensions: 6 numerical dimensions and 107 categorical one-hot dimensions. When the target one-hot vector is included, the full GAN representation has 115 dimensions.

The preprocessing dimensionality summary is shown in Appendix Table A3.

3. GAN and cGAN Architectures

All neural models were implemented and trained using PyTorch.

3.1 Standard GAN

The standard GAN follows the classical adversarial setup. The generator receives a Gaussian noise vector z of dimension 64 and maps it to a full synthetic tabular vector of dimension 115, including numerical features, categorical one-hot-like features, and the one-hot income label. The discriminator receives either a real preprocessed vector or a generated vector and outputs a single real/fake logit.

Both generator and discriminator are implemented as fully connected multilayer perceptrons. The generator uses hidden dimensions (128, 256, 256) with BatchNorm and LeakyReLU. Its output is split into numerical and non-numerical parts: tanh is applied to numerical outputs to match the $[-1, 1]$ preprocessing range, while sigmoid is applied to categorical and target outputs to keep them in $[0, 1]$. The discriminator uses hidden dimensions (256, 128, 64), LeakyReLU activations, and dropout.

The model was trained for 300 epochs using BCEWithLogitsLoss and Adam optimizer with learning rate 0.0002 and betas (0.5, 0.999). Training was repeated for seeds 42, 123, and 2026.

The full hyperparameter and final training summaries are shown in Appendix Table A4. Representative loss curves are shown in Appendix Figure A1.

3.2 Conditional GAN

The cGAN modifies the generator and discriminator by explicitly conditioning generation on the requested income label. The generator receives the concatenation of Gaussian noise and a one-hot income condition. It generates only the feature vector, not the target. The discriminator receives the feature vector concatenated with the same one-hot income condition and predicts whether the pair is real or synthetic.

For real samples, the condition is the true income label. For synthetic samples, the condition is the requested label used by the generator. After training, we generated a synthetic dataset with the same size as the training set and with the same requested label ratios as the original training set.

The cGAN uses the same optimization setup as the standard GAN: 300 epochs, BCEWithLogitsLoss, Adam optimizer, learning rate 0.0002, and betas (0.5, 0.999).

The cGAN training summaries are shown in Appendix Table A5, and representative cGAN loss curves are shown in Appendix Figure A2.

4. Main Evaluation Results

4.1 Evaluation protocol

After training, each model generated a synthetic dataset equal in size to its corresponding training set. For the cGAN, the requested income conditions were sampled to exactly match the income-label ratios of the original training set.

Before evaluation, synthetic categorical and target blocks were hard-decoded using argmax into valid one-hot representations. This was done to avoid giving the Random Forest detector an artificial advantage caused only by soft categorical outputs. Numerical features were left unchanged.

For the detection metric, the real training set and synthetic training set were split into four folds. In each fold, a Random Forest classifier was trained on three folds from each data type and tested on the remaining real and synthetic folds. The reported detection result is the average AUC over the four folds. Lower AUC is better, with 0.5 indicating that real and synthetic samples are hard to distinguish.

For the efficacy metric, we first trained a Random Forest on the real training set and evaluated it on the real test set. Then we trained another Random Forest on the synthetic training set and evaluated it on the same real test set. The efficacy score is the ratio between the synthetic-trained AUC and the real-trained AUC. Higher values are better, with 1.0 indicating that synthetic data is as useful as real data for the downstream classification task.

4.2 Results and analysis

The detection AUC was almost 1.0 for both GAN and cGAN. This means that the Random Forest detector could almost perfectly distinguish synthetic samples from real samples. Therefore, the generated samples did not fully match the real data distribution. Since the evaluation used hard-decoded categorical blocks, this result cannot be explained only by soft categorical outputs. It likely reflects deeper distributional differences, such as mismatched numerical distributions, categorical frequencies, or feature dependencies.

This is also visible in the distribution plots. In Appendix Figure A3, several numerical features show clear mismatches between real and synthetic distributions, especially education-num and hours-per-week. In Appendix Figure A4, categorical features such as occupation and workclass show that the GAN over-represents some categories while under-representing others. These visual differences are consistent with the high detection AUC.

Despite the poor detection performance, the efficacy results were much stronger. The standard GAN achieved an average efficacy ratio of 0.9133, and the cGAN achieved an average efficacy ratio of 0.9125 across the three seeds. This indicates that although the synthetic data is easy to detect, it still preserves useful predictive information for the income classification task.

This creates an important distinction between fidelity and utility. The synthetic data is not fully realistic according to the detection metric, but it remains useful for training a downstream income classifier.

Table 1: Main evaluation results averaged across seeds

Model	Detection AUC mean	Efficacy ratio mean	Synthetic-train AUC mean
GAN	0.999999	0.913256	0.825922
cGAN	0.999997	0.912549	0.825267

Table 1 shows that both models achieved high efficacy but poor detection performance.

The per-seed evaluation results are reported in Appendix Table A6. Representative visual comparisons between real and synthetic distributions are shown in Appendix Figures A3-A6.

5. Discrete Feature Generation

5.1 Why argmax blocks gradient flow

Categorical features cannot be generated by applying argmax before passing the generated sample to the discriminator. Although argmax produces a valid one-hot category in the forward pass, it is not differentiable. It maps a continuous vector of logits or probabilities into a discrete index, and small changes in the generator output usually do not change the selected category. Therefore, the derivative is zero or undefined almost everywhere. As a result, the discriminator loss cannot provide meaningful gradients back to the generator through the categorical decision, preventing the generator from learning how to adjust its categorical logits.

5.2 Compared strategies

We compared two differentiable strategies for categorical generation using the standard unconditional GAN with seed 42. In the first strategy, the generator applies a separate softmax to each categorical feature block and passes the resulting soft probability distributions directly to the discriminator. In this case, the forward pass is soft and the backward pass is fully differentiable through the softmax.

In the second strategy, the generator uses the Gumbel-Softmax reparameterization with a straight-through estimator. In the forward pass, the discriminator receives hard or near-discrete one-hot samples. In the backward pass, gradients flow through the underlying soft Gumbel-Softmax probabilities. This gives the discriminator a more realistic discrete representation while still allowing the generator to learn.

5.3 Entropy and results

To quantify how committed the generator was to categorical decisions, we computed the average entropy of the generated soft distributions before decoding for each categorical feature. We compared it to the entropy implied by the real category frequencies. A very high entropy close to the uniform maximum would indicate that the generator emits nearly uniform probabilities, while a very low entropy would indicate overly confident or collapsed categorical generation.

The softmax strategy produced extremely low generated entropy for almost all categorical features. For example, occupation had real entropy 2.4397 but generated entropy only 0.0710, and native-country had real entropy 0.6531 but generated entropy 0.0008. This suggests that the generator became overly confident and collapsed to a narrow set of categories.

The Gumbel-ST strategy produced substantially more realistic entropy values. For example, marital-status had real entropy 1.2726 and generated entropy 1.2159, race had real entropy 0.5513 and generated entropy 0.6032, and native-country had real entropy 0.6531 and generated entropy 0.6930. Some gaps remained, especially for education and workclass, but overall the categorical diversity was much better than with plain softmax.

The efficacy ratio also improved from 0.9302 with softmax to 0.9423 with Gumbel-ST. Detection AUC remained close to 1.0 for both methods, so neither representation solved the global fidelity problem. However, based on entropy and efficacy, we would keep the Gumbel-ST representation.

Table 2: Discrete generation strategy summary

Strategy	Detection AUC	Efficacy ratio	Avg generated entropy ratio
Softmax	0.999988	0.930170	0.018810
Gumbel-ST	0.999939	0.942291	0.403208

The full entropy comparison per categorical feature is shown in Appendix Table A7. Loss curves for both strategies are shown in Appendix Figure A7.

6. Mode Collapse

6.1 Collapse indicators

We defined two quantitative indicators for mode collapse. The first is categorical combination coverage, computed as the number of unique categorical-value combinations in the synthetic data divided by the number of unique categorical-value combinations in the real training data. This is suitable for Adult because many features are categorical, and mode collapse should reduce the number of generated discrete combinations.

The second indicator is the mean numeric variance ratio, computed as the average over numerical features of synthetic variance divided by real variance. If the generator collapses to a narrow set of numerical outputs, this ratio should decrease.

6.2 Inducing collapse

To deliberately induce mode collapse, we reduced the latent dimension to 4, reduced the generator capacity to a single hidden layer of size 32, and trained the discriminator five times for each generator update. This strongly unbalanced the adversarial game in favor of the discriminator.

The induced collapse was clear. The real training data contained 8,296 unique categorical combinations, while the collapsed generator produced only 27. This gives a categorical combination coverage of 0.00325. The mean numeric variance ratio was 0.4150, and the minimum numeric variance ratio was only 0.0014.

The loss curves also indicate collapse. The final discriminator score on real samples was 0.9869, while its score on fake samples was only 0.0135. The generator adversarial loss remained high at 5.9006, showing that the discriminator could easily reject generated samples.

6.3 Mitigation and analysis

As a mitigation, we added an explicit diversity penalty to the generator objective. The penalty encourages higher batch-level standard deviation in generated samples. Before running the experiment, we predicted that if the mitigation worked, categorical combination coverage, mean numeric variance ratio, and batch diversity should increase.

The mitigation did not recover the collapsed modes. Categorical coverage decreased from 0.00325 to 0.00277, and the mean numeric variance ratio decreased from 0.4150 to 0.2939. The final batch diversity score also slightly decreased from 0.0454 to 0.0439.

This negative result suggests that the diversity penalty was too weak or too generic to overcome the strong discriminator and the very limited generator capacity. In particular, a batch standard-deviation penalty encourages general spread in the output vector, but it does not directly encourage coverage of realistic categorical combinations. Therefore, the mitigation did not help in this setup.

Table 3: Mode collapse and mitigation results

Experiment	Unique synthetic combinations	Coverage	Mean numeric variance ratio	Batch diversity
Collapsed	27	0.003255	0.414963	0.045376
Diversity penalty	23	0.002772	0.293929	0.043899

The detailed mode-collapse results are shown in Appendix Table A8, and the loss/diversity curves are shown in Appendix Figure A8.

7. Open-ended Contribution: Feature Matching GAN

7.1 Modification and motivation

For the open-ended contribution, we modified the standard GAN training objective by adding feature matching to the generator loss. Instead of optimizing only the discriminator’s final real/fake output, the generator was also trained to match the mean intermediate discriminator representation of real samples.

The motivation was based on the main evaluation results. The baseline GAN achieved relatively high efficacy but almost perfect detection AUC, meaning that the synthetic data was useful for prediction but still easily distinguishable from real data. Feature matching targets this limitation by giving the generator a smoother distribution-level signal rather than relying only on the discriminator’s final binary decision.

7.2 Prediction before running

Before running the experiment, we predicted that feature matching would reduce Detection AUC, because synthetic samples should become less distinguishable from real samples. We also predicted that Efficacy would remain similar or improve, since the feature-level objective should preserve useful predictive structure.

7.3 Results and analysis

The prediction was partially supported. Feature Matching improved the efficacy ratio from 0.9237 to 0.9491, meaning that the synthetic data became more useful for downstream income prediction. However, Detection AUC only changed from 0.999999 to 0.999995, so the generated samples were still almost perfectly distinguishable from real samples.

This suggests that feature matching improved task-relevant structure but did not solve the fidelity problem. A possible explanation is that matching only the mean discriminator representation is insufficient for complex tabular data with many categorical dependencies. In addition, the feature matching loss dominated the generator objective: the final generator total loss was 60.7498, while the adversarial part was only 3.3646 and the feature matching loss was 5.7385 with weight 10.0.

Therefore, the modification was useful for efficacy but insufficient for making the synthetic data fully realistic. This is a partial positive result with an important limitation.

Table 4: Feature Matching GAN vs baseline

Model	Detection AUC	Synthetic-train AUC	Efficacy ratio
Baseline GAN	0.999999	0.838122	0.923671
Feature Matching GAN	0.999995	0.861217	0.949124

The Feature Matching training curves are shown in Appendix Figure A9.

8. Conclusion

Overall, the experiments show a consistent gap between fidelity and utility. The GAN and cGAN synthetic datasets were almost perfectly detectable by a Random Forest, indicating that they did not fully match the real Adult distribution. However, their efficacy ratios were around 0.91, showing that they preserved meaningful predictive information for the income task.

The discrete-feature experiments showed that categorical representation is a central issue in tabular GANs. Plain softmax allowed gradient flow but produced very low categorical entropy, while Gumbel-ST produced more realistic categorical diversity and higher efficacy. Therefore, Gumbel-ST is the preferred representation among the tested options.

The mode-collapse experiment successfully induced severe collapse and showed that a simple diversity penalty was not sufficient to recover the lost modes. The open-ended Feature Matching GAN improved efficacy but did not meaningfully reduce detection AUC. Future improvements should therefore focus on stronger distribution-matching objectives, better categorical modeling, or architectures designed specifically for tabular data.

Appendix

Appendix Table A1 — Dataset summary

Item	Value
Rows	32,561
Columns	15
Numerical features	6
Categorical features	8
Target	income
<=50K count	24,720
>50K count	7,841
Missing workclass	1,836
Missing occupation	1,843
Missing native-country	583

Table A1: Adult dataset summary, feature types, target distribution, and missing values.

Appendix Table A2 — Train/test split summary

Seed	Split	<=50K count	>50K count	<=50K ratio	>50K ratio
Full	Full	24720	7841	0.759190	0.240810
42	Train	19775	6273	0.759175	0.240825
42	Test	4945	1568	0.759251	0.240749
123	Train	19775	6273	0.759175	0.240825
123	Test	4945	1568	0.759251	0.240749
2026	Train	19775	6273	0.759175	0.240825
2026	Test	4945	1568	0.759251	0.240749

Table A2: Stratified 80/20 train-test splits across the three random seeds.

Appendix Table A3 — Preprocessing dimensions

Seed	Numeric dim	Categorical one-hot dim	Feature dim	Target dim	GAN dim
42	6	107	113	2	115
123	6	107	113	2	115
2026	6	107	113	2	115

Table A3: Preprocessed representation dimensions for all train-test splits.

Appendix Table A4 — Standard GAN training summary

Seed	D loss	G loss	D(real)	D(fake)
42	0.4247	2.9157	0.8693	0.1358
123	0.3790	3.0773	0.8846	0.1187
2026	0.4016	3.0070	0.8761	0.1264

Table A4: Final standard GAN training metrics after 300 epochs.

Appendix Table A5 — cGAN training summary

Seed	D loss	G loss	D(real condition)	D(fake condition)
42	0.4627	2.9939	0.8534	0.1491
123	0.4194	3.2587	0.8686	0.1346
2026	0.4843	3.1302	0.8425	0.1565

Table A5: Final cGAN training metrics after 300 epochs.

Appendix Table A6 — Per-seed evaluation

Model	Seed	Detection AUC	Real AUC	Synthetic AUC	Efficacy
GAN	42	0.999999	0.907381	0.838122	0.923671
cGAN	42	0.999993	0.907381	0.834680	0.919878
GAN	123	1.000000	0.901861	0.790337	0.876340
cGAN	123	0.999999	0.901861	0.806568	0.894337
GAN	2026	1.000000	0.903751	0.849306	0.939757
cGAN	2026	0.999999	0.903751	0.834552	0.923432

Table A6: Detection and efficacy metrics per seed for GAN and cGAN.

Appendix Table A7 — Entropy comparison

Feature	Real entropy	Softmax entropy	Gumbel-ST entropy
work class	1.1481	0.0282	0.5421
education	2.0373	0.0881	0.5154
marital status	1.2726	0.0468	1.2159
occupation	2.4397	0.0710	1.8585
relationship	1.4907	0.0513	1.1338
race	0.5513	0.0064	0.6032
sex	0.6346	0.0181	0.5231
native country	0.6531	0.0008	0.6930
income	0.5520	0.0208	0.2415

Table A7: Real and generated categorical entropy before decoding for Softmax and Gumbel-ST strategies.

Appendix Table A8 — Mode collapse

Experiment	Real unique combinations	Synthetic unique combinations	Coverage	Mean numeric variance ratio	Min variance ratio	Max variance ratio
Collapsed	8296	27	0.003255	0.414963	0.001438	1.305976
Diversity penalty	8296	23	0.002772	0.293929	0.000008	0.740153

Table A8: Mode collapse indicators before and after diversity-penalty mitigation.

Appendix Table A9 — Feature Matching

Model	Detection AUC	Detection std	Real AUC	Synthetic AUC	Efficacy ratio
Baseline GAN	0.999999	0.000001	0.907381	0.838122	0.923671
Feature Matching GAN	0.999995	0.000004	0.907381	0.861217	0.949124

Table A9: Feature Matching GAN compared with the unmodified baseline GAN.

Figure A1 — Standard GAN loss curves

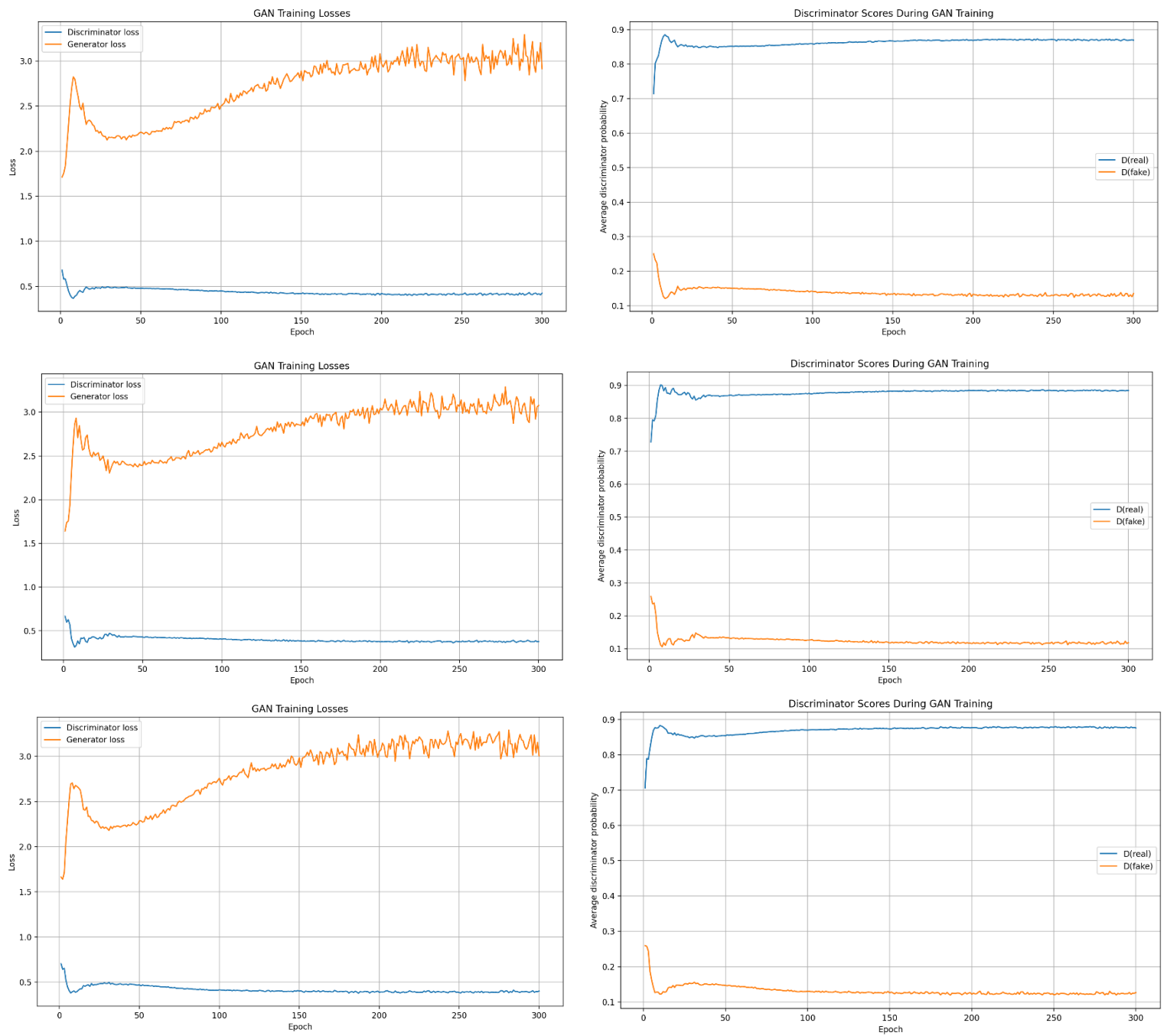


Figure A1: Standard GAN training curves for seeds 42, 123 and 2026. Loss fluctuations are expected in GAN training.

Figure A2 — cGAN loss curves

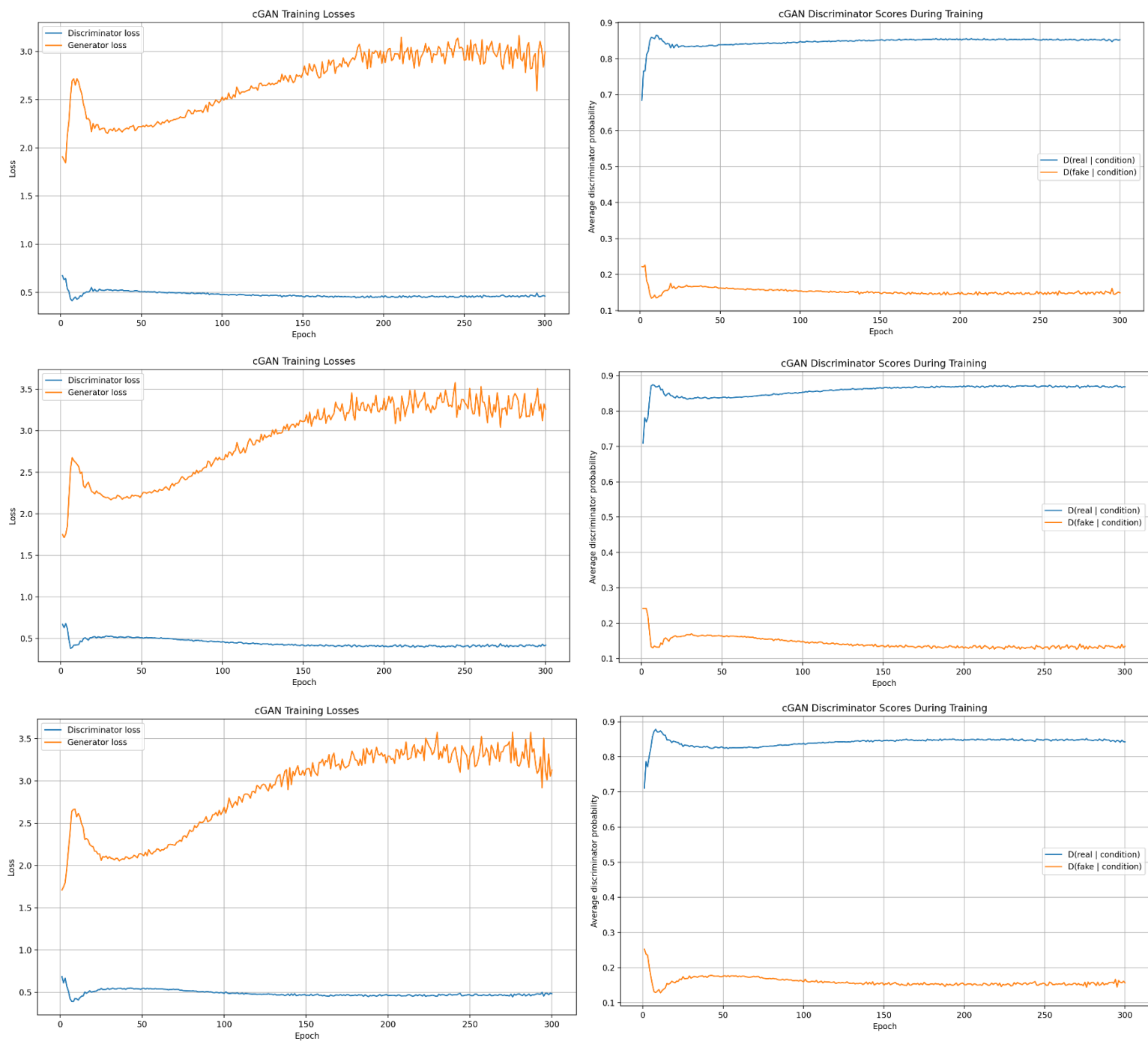


Figure A2: cGAN training curves for seeds 42, 123 and 2026.

Figure A3 — Numeric distributions

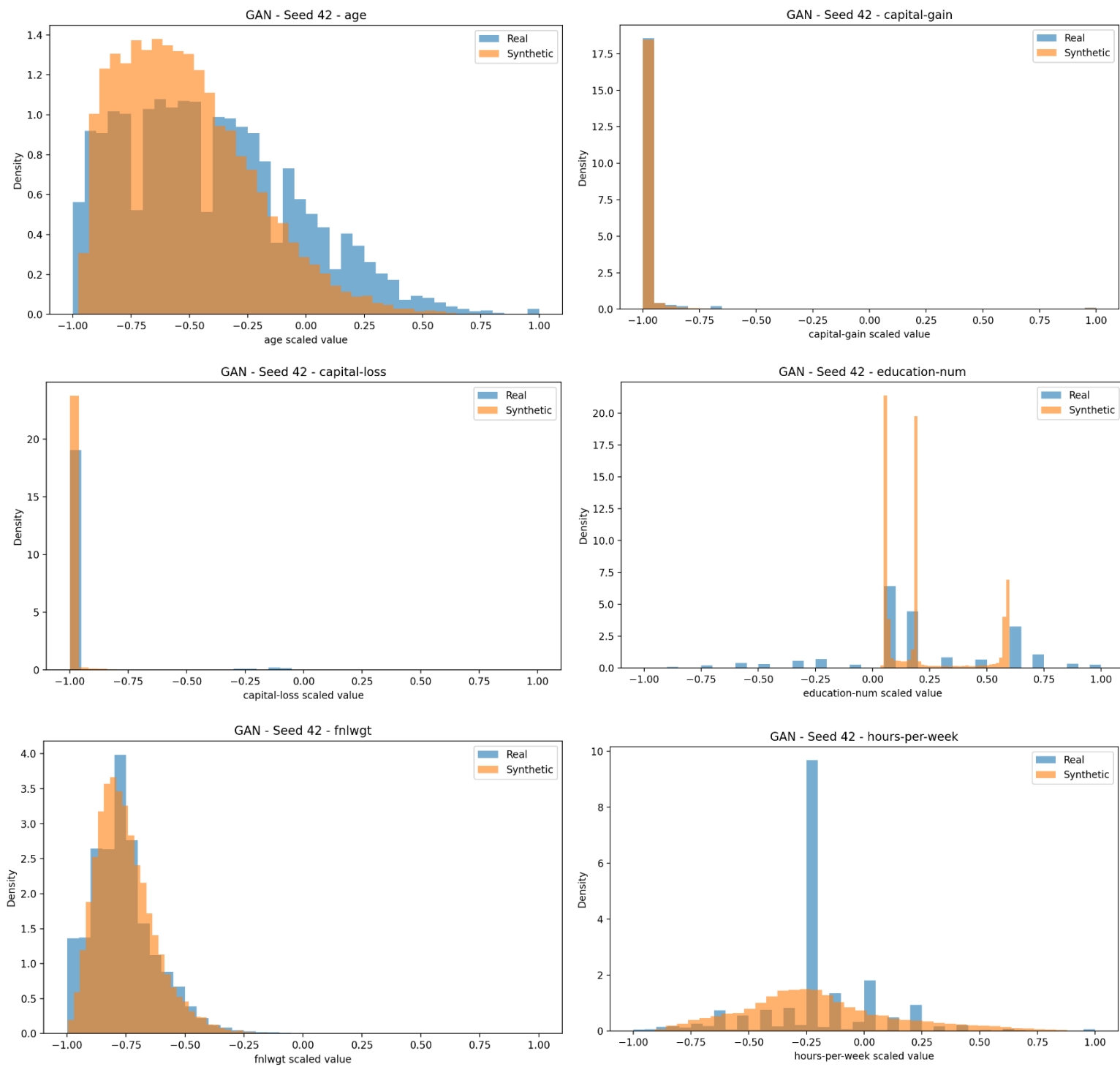


Figure A3: Real vs. synthetic numerical feature distributions for the standard GAN, seed 42.

Figure A4 — Categorical distributions

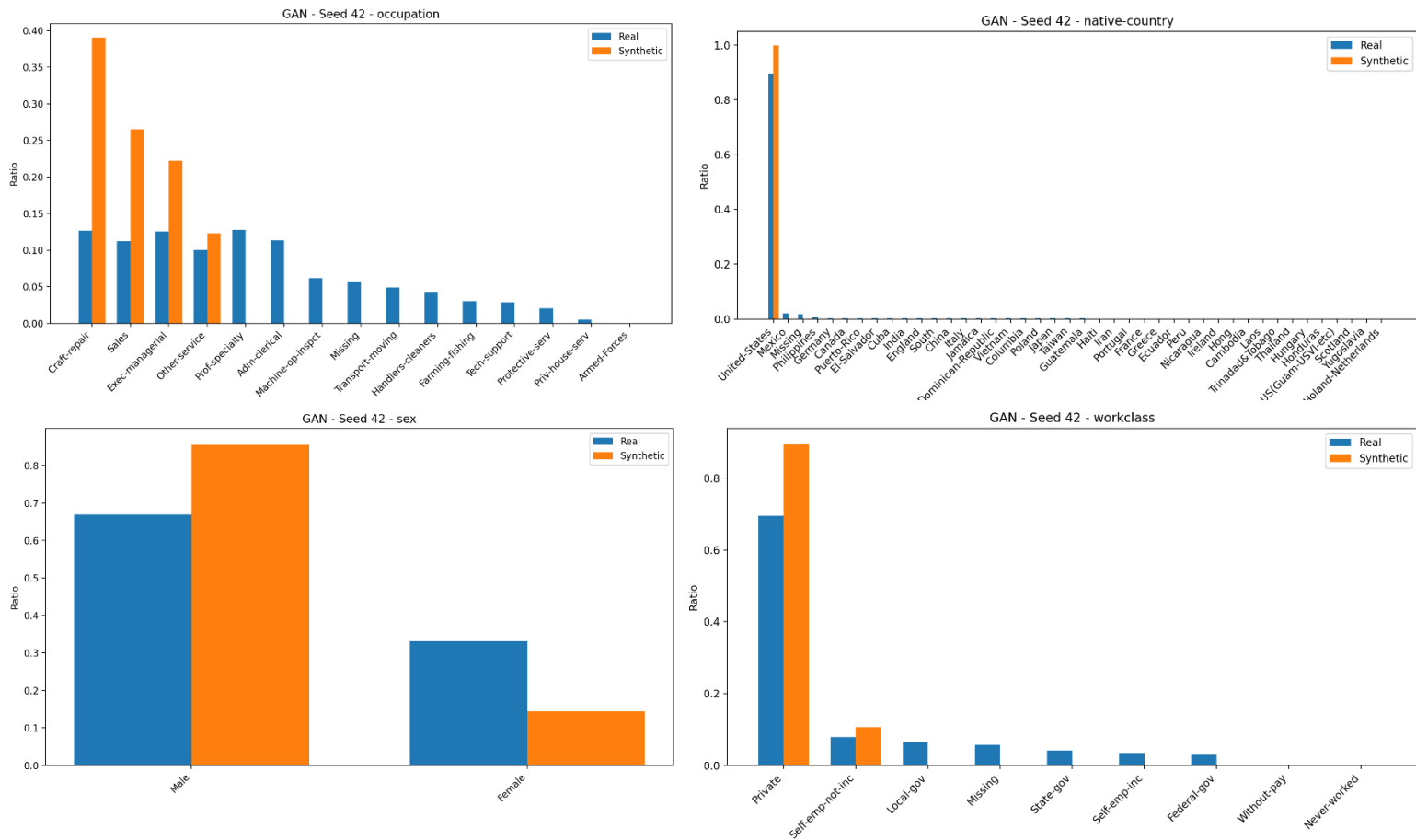


Figure A4: Real vs. synthetic categorical feature distributions for the standard GAN, seed 42.

Figure A5 — Correlation matrices

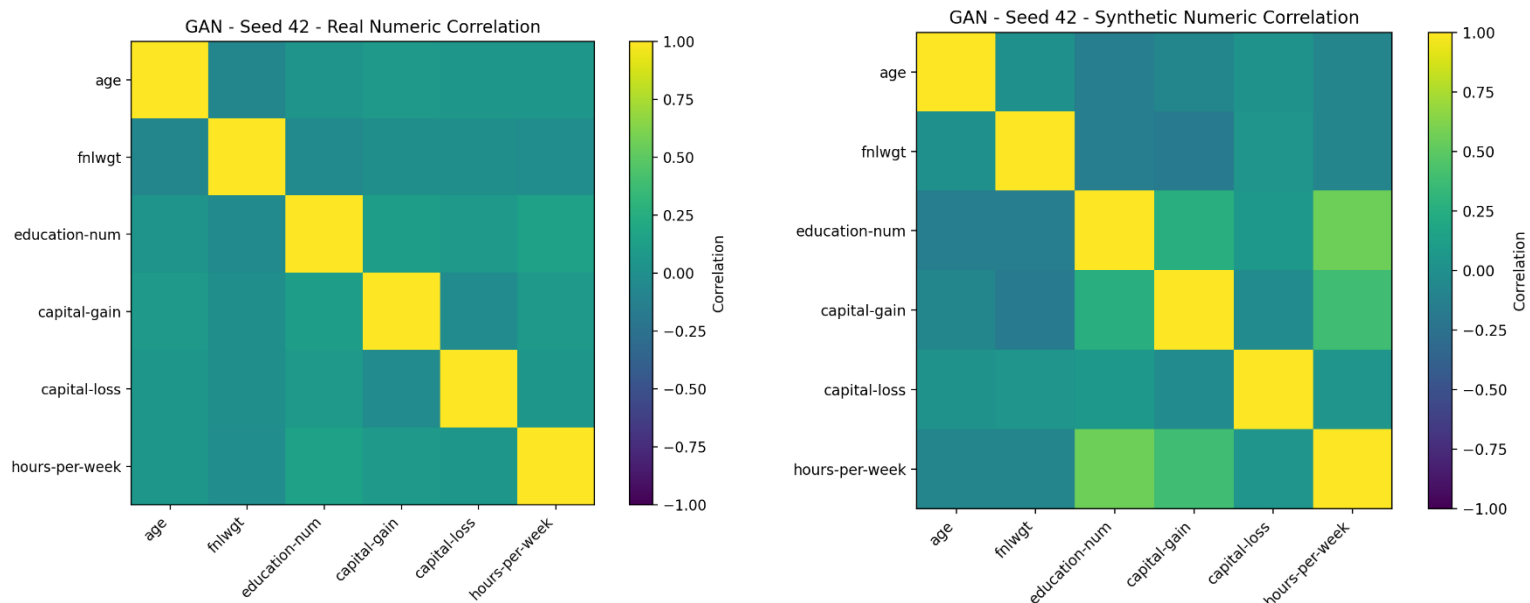


Figure A5: Real and synthetic numerical correlation matrices for the standard GAN, seed 42.

Figure A6 — Target distribution for cGAN

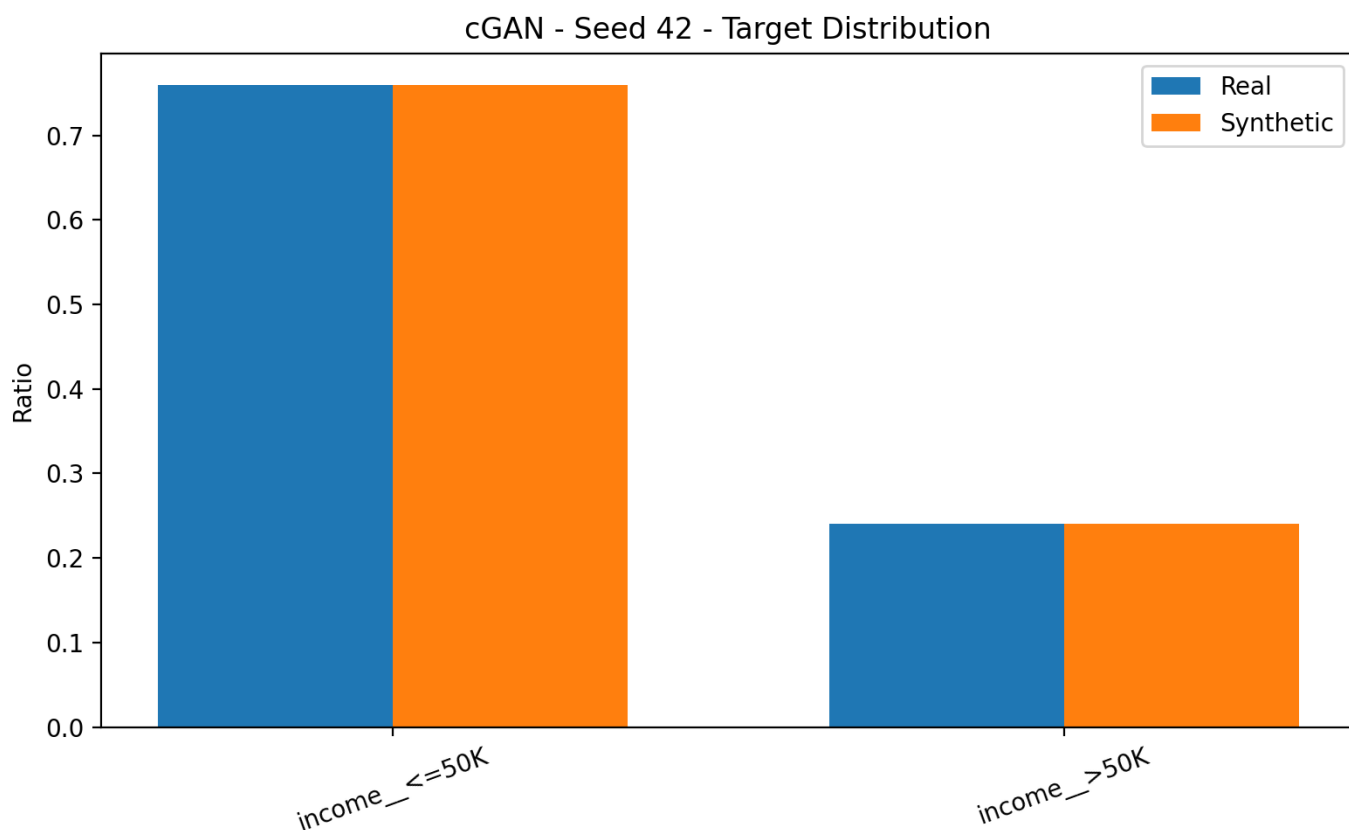


Figure A6: Real vs. synthetic target distribution for cGAN. The requested cGAN label ratios match the original training set.

Figure A7 — Discrete feature strategies

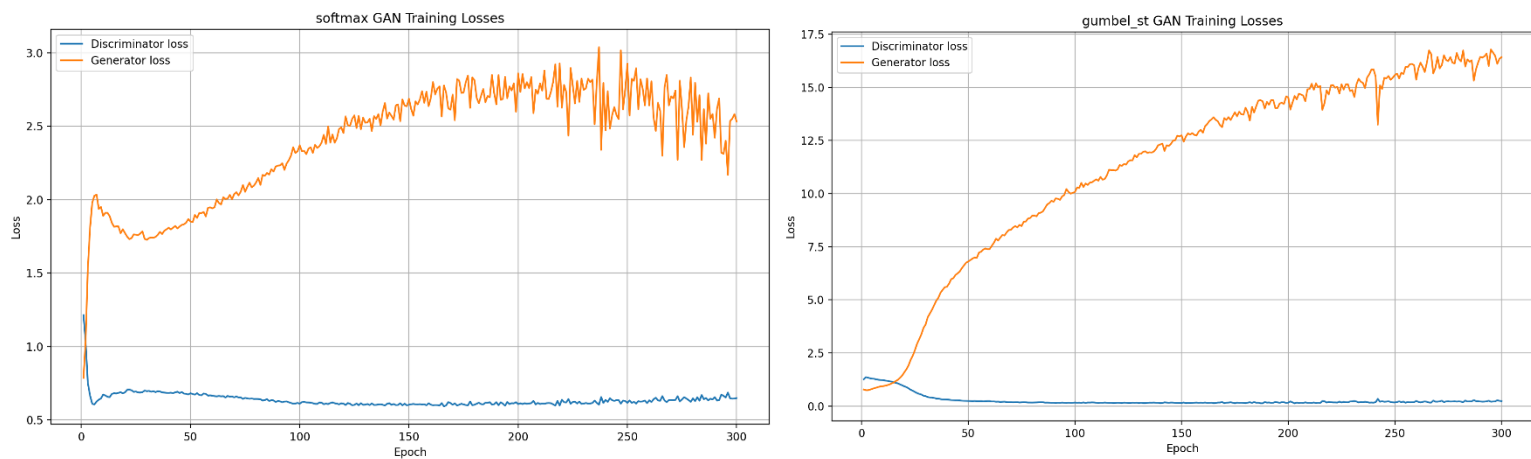


Figure A7: Training loss curves for Softmax and Gumbel-ST categorical generation strategies.

Figure A8 — Mode collapse

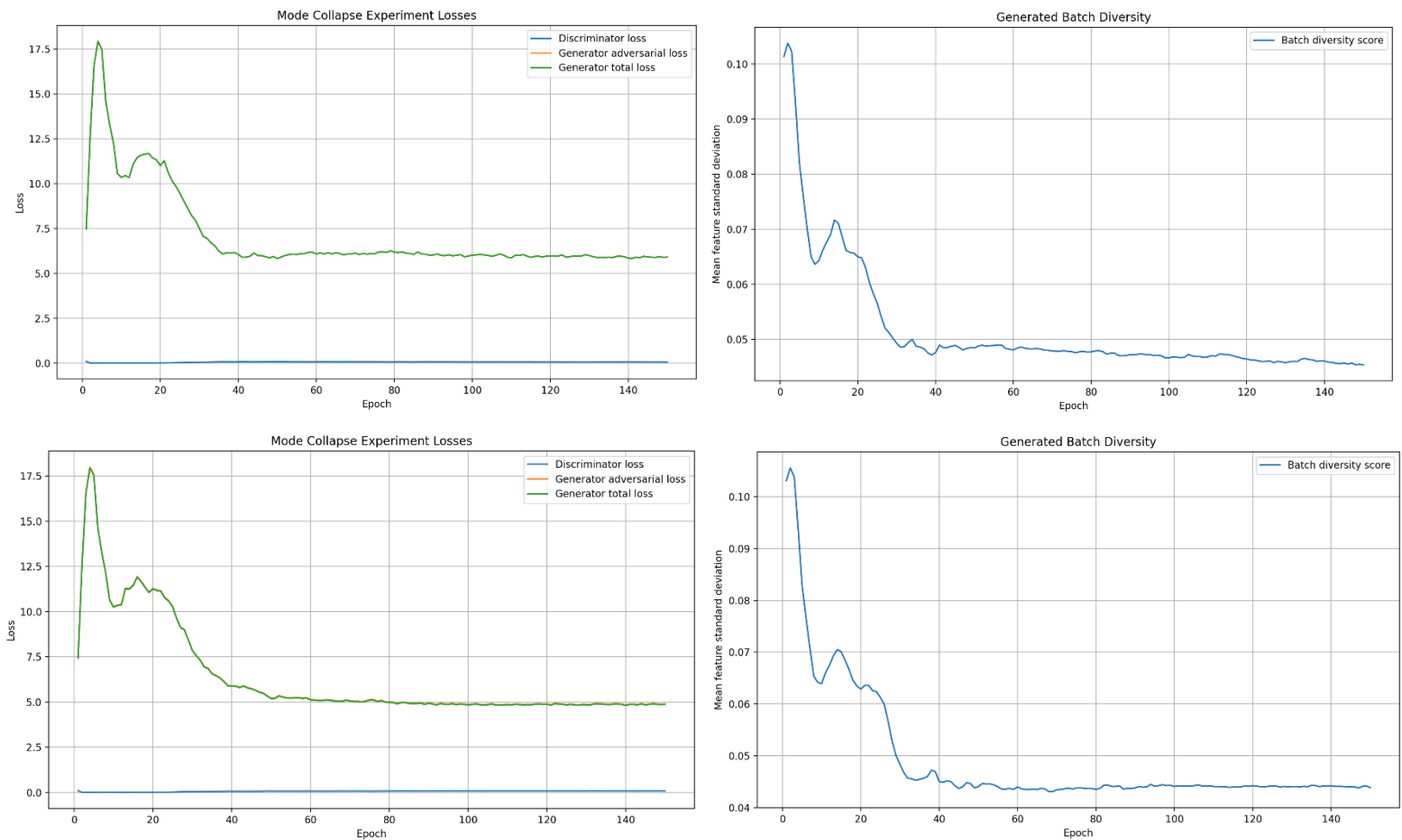


Figure A8: Mode collapse and diversity-penalty mitigation training curves.

Figure A9 — Feature Matching GAN

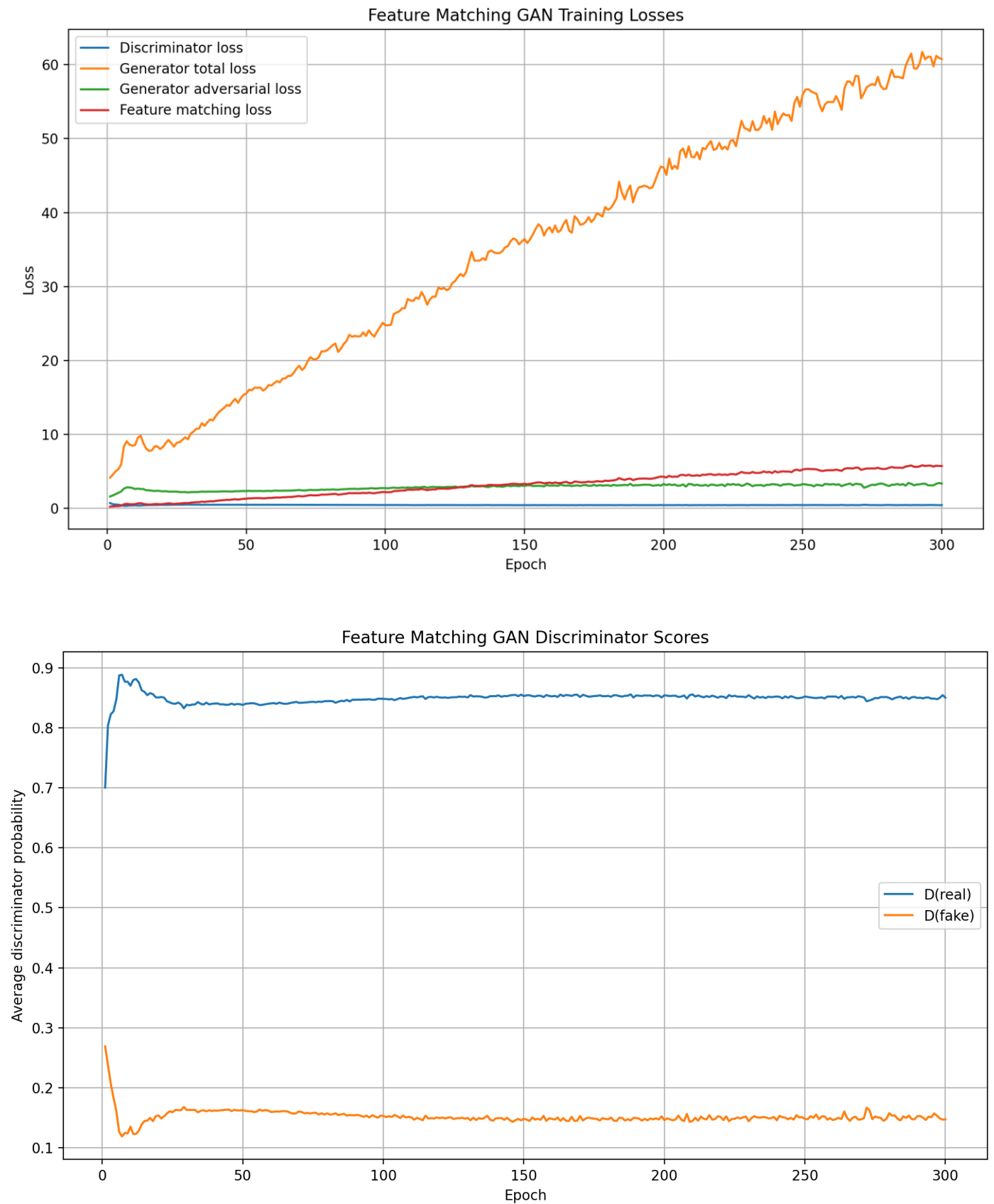


Figure A9: Feature Matching GAN training curves.